

SEO — Remaining TODO

Gouder Haithem Portfolio · Next.js (App Router) · Generated 2026-06-27 · everything is DB-backed (Prisma) + admin-managed

✓ Already done this session

Sitemap revalidation — added `revalidatePath('/sitemap.xml')` to the three admin revalidate helpers (`blog/actions.ts` , `projects/actions.ts` , `services/actions.ts`). Now when you publish or edit content in the admin panel, `/sitemap.xml` refreshes immediately, so Google discovers new posts/projects without waiting for a rebuild.

The three items below are still open. None of them change your data flow — they are all dynamic-safe (they read from the database at request time, exactly like your pages already do).

1. Social share images (OG images) High impact

WHAT IT IS

The "OG image" (Open Graph image) is the preview picture shown when a link is pasted into WhatsApp, LinkedIn, X, Facebook, Slack, or iMessage — the big rectangle above the title in the link card.

THE PROBLEM TODAY

Your posts/projects use the uploaded image, which is an `.svg` file. **Social platforms do not render SVG**, so shared links currently show a card with *no image* — just text. Weak first impression for a portfolio at the exact moment someone shares your work.

THE FIX — GENERATE THE IMAGE AUTOMATICALLY

Next.js has a special file, `opengraph-image.tsx`, that renders a real **1200×630 PNG on the fly** for each page — built from the post's title pulled straight from your database. Nothing to upload, works for every post forever, renders everywhere.

Building a CRM clients actually use

Gouder Haithem · Software Engineer

← post.title from DB

← your brand

SIMPLE EXAMPLE

Create `app/blog/[slug]/opengraph-image.tsx` :

```
// Next renders this React tree to a PNG automatically.
import { ImageResponse } from 'next/og'
import { getPostBySlug } from '@lib/blog'

export const size = { width: 1200, height: 630 }
export const contentType = 'image/png'
```

```
export default async function Image({ params }) {
  const post = await getPostBySlug(params.slug)
  return new ImageResponse(
    (
      <div style={{
        width: '100%', height: '100%', display: 'flex',
        flexDirection: 'column', justifyContent: 'center',
        padding: 80, background: '#0b0b0f', color: '#fff',
      }}>
        <div style={{ fontSize: 64, fontWeight: 700 }}>{post?.title}</div>
        <div style={{ fontSize: 28, marginTop: 24, color: '#9aa' }}>
          Gouder Haithem · Software Engineer
        </div>
      </div>
    ),
    { ...size }
  )
}
```

Then in the page's `generateMetadata` you can drop the manual `openGraph.images` line — Next auto-detects `opengraph-image.tsx` and wires it up. Repeat the same file for `app/projects/[slug]/opengraph-image.tsx`.

Why generated, not "just upload PNGs": images come from your admin panel. Forcing yourself to upload a perfect 1200×630 PNG for every post is easy to forget and easy to get wrong. The generated approach is set-once and automatic.

Approach	Works on social?	Effort per post
Current SVG upload	✗ no image	—
Force PNG/JPG in admin	✓ yes	manual every time
opengraph-image.tsx (generated)	✓ always	zero

Note: this only affects the share-preview image. The image shown on the actual page still uses whatever you upload in the admin — unchanged.

2. Article / Project structured data (JSON-LD) Medium impact

WHAT IT IS

JSON-LD is a hidden block of machine-readable facts that tells Google *exactly* what a page is — "this is an Article titled X, by Gouder Haithem, published on date Y." You already have this for your *person/business* in `app/layout.tsx` (the `Person` / `ProfessionalService` / `WebSite` graph), but it is missing on individual blog posts and projects.

WHY IT HELPS

Pages with `BlogPosting` / `CreativeWork` structured data can earn **rich results** in Google — author, date, and headline shown in an enhanced listing instead of a plain blue link. Posts still rank fine without it; this just makes them eligible for the richer presentation.

SIMPLE EXAMPLE

Inside `app/blog/[slug]/page.tsx`, render a script tag from the DB row:

```
const jsonLd = {
  '@context': 'https://schema.org',
  '@type': 'BlogPosting',
  headline: post.title,
  description: post.excerpt,
  datePublished: post.date,
  author: { '@type': 'Person', name: post.author.name },
  image: `https://gouderhaithem.com/blog/${post.slug}/opengraph-image`,
}

return (
  <>
    <script
      type="application/ld+json"
      dangerouslySetInnerHTML={{ __html: JSON.stringify(jsonLd) }}
    />
    { /* ...rest of the page... */ }
  </>
)
```

Bonus: point the `image` field at the generated `opengraph-image` from item 1 — one branded image serves both Google rich results and social cards.

Do the same on `app/projects/[slug]/page.tsx` using `@type: 'CreativeWork'` (or `'SoftwareApplication'`).

3. Block /admin in robots.ts Low / tidy

WHAT IT IS

`app/robots.ts` tells crawlers what they may visit. Yours currently allows everything and never mentions `/admin`.

IS /ADMIN EXPOSED?

No — `middleware.ts` already redirects anyone without a valid session away from `/admin`. So this is purely cosmetic: an explicit `disallow` stops crawlers from even attempting the private area.

SIMPLE EXAMPLE

```
export default function robots() {
  return {
    rules: {
      userAgent: '*',
      allow: '/',
      disallow: '/admin', // <-- add this line
    },
    sitemap: 'https://gouderhaithem.com/sitemap.xml',
  }
}
```

```
}  
}
```

Priority order

#	Item	Impact	Effort
1	OG images (<code>opengraph-image.tsx</code>)	High — fixes broken social previews	~30 min, 2 files
2	BlogPosting / CreativeWork JSON-LD	Medium — rich results eligibility	~20 min, 2 files
3	<code>disallow: /admin</code> in robots.ts	Low — tidiness	~1 min, 1 line

Reference files in this repo: `app/layout.tsx` (existing JSON-LD) · `app/sitemap.ts` · `app/robots.ts` · `middleware.ts` · `app/blog/[slug]/page.tsx` · `app/projects/[slug]/page.tsx` · `lib/blog.ts` & `lib/projects.ts` (DB queries).